Use this shell variable in examples:

```shell
CID=test-cid-001
```

1) Mock backend cookbook

These endpoints simulate downstream ERP, CRM, and Shipping behavior.

1.1 Health

```shell
curl -s http://localhost:8081/health
```

What happens beneath:

- Express returns uptime and the current fail/slow state for erp, crm, and shipping.

Expected result:

- status: "OK"
- current state
- uptime info

1.2 Inspect admin state

```shell
curl -s http://localhost:8081/admin/state
```

What happens beneath:

- Returns the runtime flags controlling failure and latency injection.

Expected result:

- JSON showing:
    - erp.fail, erp.slowMs
    - crm.fail, crm.slowMs
    - shipping.fail, shipping.slowMs

1.3 Put ERP in fail mode

```shell
curl -s -X POST http://localhost:8081/admin/erp/fail \
  -H 'Content-Type: application/json' \
  -d '{"enabled": true}'
```

What happens beneath:

- Sets state.erp.fail = true.

Expected result:

- status: "OK"
- target erp
- fail: true

To disable:

```shell
curl -s -X POST http://localhost:8081/admin/erp/fail \
  -H 'Content-Type: application/json' \
  -d '{"enabled": false}'
```

1.4 Put CRM in fail mode

```shell
curl -s -X POST http://localhost:8081/admin/crm/fail \
  -H 'Content-Type: application/json' \
  -d '{"enabled": true}'
```

What happens beneath:

- Sets state.crm.fail = true.

Expected result:

- CRM will start returning HTTP 503 on /ping, /normal, /slow.

Disable:

```shell
curl -s -X POST http://localhost:8081/admin/crm/fail \
```

```
  -H 'Content-Type: application/json' \
  -d '{"enabled": false}'
```

1.5 Put Shipping in fail mode

```shell
curl -s -X POST http://localhost:8081/admin/shipping/fail \
  -H 'Content-Type: application/json' \
  -d '{"enabled": true}'
```

What happens beneath:

- Sets state.shipping.fail = true.

Expected result:

- Any async shipping delivery coming from MI will fail and be retried by the message processor.

Disable:

```shell
curl -s -X POST http://localhost:8081/admin/shipping/fail \
  -H 'Content-Type: application/json' \
  -d '{"enabled": false}'
```

1.6 Change ERP slow latency

```shell
curl -s -X POST http://localhost:8081/admin/erp/slow \
  -H 'Content-Type: application/json' \
  -d '{"ms": 12000}'
```

What happens beneath:

- Sets ERP slow endpoint latency to 12s.
- Only affects /erp/slow, not /erp/normal.

Expected result:

- slowMs: 12000

1.7 Change CRM slow latency

```shell
curl -s -X POST http://localhost:8081/admin/crm/slow \
  -H 'Content-Type: application/json' \
  -d '{"ms": 7000}'
```

What happens beneath:

- Sets CRM /slow delay to 7s.

Expected result:

- slowMs: 7000

1.8 Change Shipping slow latency

```shell
curl -s -X POST http://localhost:8081/admin/shipping/slow \
  -H 'Content-Type: application/json' \
  -d '{"ms": 15000}'
```

What happens beneath:

- Sets shipping /slow delay to 15s.
- This matters because MI shipping endpoint points to /shipping/slow.

Expected result:

- slowMs: 15000

1.9 ERP ping

```shell
curl -s -H "x-correlation-id: $CID" http://localhost:8081/erp/ping
```

What happens beneath:

- Quick GET health-style check for ERP mock.

Expected result:

- status: "OK"

- echoed cid

If ERP is in fail mode:

- HTTP 503
- JSON error payload

1.10 CRM ping

```shell
curl -s -H "x-correlation-id: $CID" http://localhost:8081/crm/ping
```

What happens beneath:

- Quick GET check for CRM mock.

Expected result:

- status: "OK"

1.11 Shipping ping

```shell
curl -s -H "x-correlation-id: $CID" http://localhost:8081/shipping/ping
```

What happens beneath:

- Quick GET check for Shipping mock.

Expected result:

- status: "OK"

1.12 ERP normal call

```shell
curl -s -X POST http://localhost:8081/erp/normal \
  -H 'Content-Type: application/json' \
  -H "x-correlation-id: $CID" \
  -d '{
    "orderId": "ORD-1001",
```

```
  "amount": 12000,
  "customerId": "CUST-1"
}'
```

What happens beneath:

- ERP mock synthesizes:
  - erpOrderRef
  - creditStatus
  - warehouse

Expected result:

- immediate JSON response
- for amount 12000, likely:
  - creditStatus: "APPROVED"
  - warehouse: "WH-PREMIUM"

For amount above 50000, ERP returns REQUIRES_APPROVAL.

1.13 CRM normal call

```shell
curl -s -X POST http://localhost:8081/crm/normal \
  -H 'Content-Type: application/json' \
  -H "x-correlation-id: $CID" \
  -d '{
    "orderId": "ORD-1002",
    "amount": 15000,
    "customerId": "CUST-9"
  }'
```

What happens beneath:

- CRM mock synthesizes:
  - customerTier
  - loyaltyId
  - flaggedCustomer

Expected result:

- likely:
  - customerTier: "GOLD"
  - loyaltyId: "LOY-CUST-9"

    - flaggedCustomer: false

For amount above 100000, flaggedCustomer becomes true.

1.14 Shipping normal call

```shell
curl -s -X POST http://localhost:8081/shipping/normal \
  -H 'Content-Type: application/json' \
  -H "x-correlation-id: $CID" \
  -d '{
    "orderId": "ORD-1003",
    "amount": 8000,
    "premium": false
  }'
```

What happens beneath:

- Shipping mock returns:
    - carrier
    - ETA
    - tracking seed

Expected result:

- likely:
    - carrier: "ECONOMY_SHIP"
    - etaDays: 3

For amount above 10000, it returns FAST_SHIP and etaDays: 1.

1.15 ERP slow call

```shell
time curl -s -X POST http://localhost:8081/erp/slow \
  -H 'Content-Type: application/json' \
  -H "x-correlation-id: $CID" \
  -d '{
    "orderId": "ORD-1004",
    "amount": 9000
  }'
```

What happens beneath:

- Waits for configured ERP slowMs, then returns normal ERP-like output.

Expected result:

- response delayed by erp.slowMs

1.16 CRM slow call

```shell
time curl -s -X POST http://localhost:8081/crm/slow \
  -H 'Content-Type: application/json' \
  -H "x-correlation-id: $CID" \
  -d '{
    "orderId": "ORD-1005",
    "amount": 20000,
    "customerId": "CUST-77"
  }'
```

What happens beneath:

- waits, then responds

Expected result:

- delayed response
- CRM enrichment JSON

1.17 Shipping slow call

```shell
time curl -s -X POST http://localhost:8081/shipping/slow \
  -H 'Content-Type: application/json' \
  -H "x-correlation-id: $CID" \
  -d '{
    "orderId": "ORD-1006",
    "amount": 25000,
    "premium": true
  }'
```

What happens beneath:

- waits for shipping slowMs
- returns shipping result

Expected result:

- delayed response
- shipping result JSON

2) Micro Integrator cookbook

These are the main integration APIs.

2.1 MI readiness

```shell
curl -s http://localhost:9201/healthz
```

What happens beneath:

- Hits MI runtime health endpoint.

Expected result:

- readiness-style health JSON or status response from MI runtime

2.2 MI liveness

```shell
curl -s http://localhost:9201/liveness
```

What happens beneath:

- Hits MI runtime liveness endpoint.

Expected result:

- liveness-style response

2.3 POST /v1/orders — happy path

```shell
curl -s -X POST http://localhost:8290/v1/orders \
  -H 'Content-Type: application/json' \
  -H "x-correlation-id: $CID" \
  -d '{
```

```
  "orderId": "ORD-V1-001",
  "amount": 12000,
  "customerId": "CUST-100",
  "channel": "web"
}'
```

What happens beneath:

- MI validates orderId and amount
- ensures CORRELATION_ID
- calls ERP synchronously
- restores original payload
- calls CRM synchronously
- builds an immediate aggregated response
- stores a shipping message asynchronously in ShippingMS
- background ShippingMP later forwards it to mock shipping /shipping/slow

Expected result:

- immediate HTTP 200 response with:
    - status: "ACCEPTED"
    - correlationId
    - order
    - erp
    - crm

Important:

- shipping result is **not** included in the response
- shipping happens later in background

2.4 POST /v1/orders — validation failure

```shell
curl -s -X POST http://localhost:8290/v1/orders \
  -H 'Content-Type: application/json' \
  -d '{
    "amount": 12000
  }'
```

What happens beneath:

- Order_Validate_Enrich sees missing orderId

Expected result:

- HTTP 400
- JSON:
    - status: "INVALID"
    - message about orderId and amount are required
    - generated correlation ID

2.5 POST /v1/orders — ERP failure path

First force ERP failure:

```shell
curl -s -X POST http://localhost:8081/admin/erp/fail \
  -H 'Content-Type: application/json' \
  -d '{"enabled": true}'
```

Then call v1:

```shell
curl -s -i -X POST http://localhost:8290/v1/orders \
  -H 'Content-Type: application/json' \
  -H "x-correlation-id: $CID" \
  -d '{
    "orderId": "ORD-V1-ERP-FAIL",
    "amount": 5000,
    "customerId": "CUST-101",
    "channel": "mobile"
  }'
```

What happens beneath:

- MI calls ERP endpoint
- ERP returns 503
- Order_Orchestrate_ERP_CRM captures HTTP status
- fail-fast branch invokes CommonFaultHandler
- CRM is not called
- shipping is not stored

Expected result:

- HTTP 503
- JSON:
    - status: "ERROR"

    - message: "Integration failure"
    - detail: "ERP call failed with HTTP 503"

Reset ERP:

```shell
curl -s -X POST http://localhost:8081/admin/erp/fail \
  -H 'Content-Type: application/json' \
  -d '{"enabled": false}'
```

2.6 POST /v1/orders — CRM failure path

Force CRM failure:

```shell
curl -s -X POST http://localhost:8081/admin/crm/fail \
  -H 'Content-Type: application/json' \
  -d '{"enabled": true}'
```

Then call v1:

```shell
curl -s -i -X POST http://localhost:8290/v1/orders \
  -H 'Content-Type: application/json' \
  -H "x-correlation-id: $CID" \
  -d '{
    "orderId": "ORD-V1-CRM-FAIL",
    "amount": 9000,
    "customerId": "CUST-102",
    "channel": "store"
  }'
```

What happens beneath:

- ERP succeeds
- MI restores original payload
- CRM returns 503
- MI fail-fast logic returns centralized fault JSON
- shipping is not stored

Expected result:

- HTTP 503

- JSON fault response
- detail mentions CRM HTTP 503

Reset CRM:

```shell
curl -s -X POST http://localhost:8081/admin/crm/fail \
  -H 'Content-Type: application/json' \
  -d '{"enabled": false}'
```

2.7 POST /v2/orders — non-premium happy path

```shell
curl -s -X POST http://localhost:8290/v2/orders \
  -H 'Content-Type: application/json' \
  -H "x-correlation-id: $CID" \
  -d '{
    "orderId": "ORD-V2-001",
    "amount": 5000,
    "customerId": "CUST-200",
    "channel": "web"
  }'
```

What happens beneath:

- same validation and sync ERP/CRM orchestration as v1
- IS_PREMIUM becomes false
- shipping async step is skipped

Expected result:

- immediate accepted response with ERP and CRM data
- no shipping queue/store action

2.8 POST /v2/orders — premium happy path

```shell
curl -s -X POST http://localhost:8290/v2/orders \
  -H 'Content-Type: application/json' \
  -H "x-correlation-id: $CID" \
  -d '{
    "orderId": "ORD-V2-002",
    "amount": 25000,
```

```
  "customerId": "CUST-201",
  "channel": "web"
}'
```

What happens beneath:

- validation + ERP + CRM
- because amount > 10000, IS_PREMIUM=true
- MI stores async shipping message to ShippingMS
- ShippingMP forwards later to shipping mock

Expected result:

- immediate accepted response
- same response shape as v1
- async shipping happens only because premium

2.9 POST /v2/orders — validation failure

```shell
curl -s -X POST http://localhost:8290/v2/orders \
  -H 'Content-Type: application/json' \
  -d '{
    "orderId": "",
    "amount": 0
  }'
```

What happens beneath:

- validation rejects missing/blank values

Expected result:

- HTTP 400
- JSON invalid response

2.10 POST /review — direct MI review call

```shell
curl -s -X POST http://localhost:8290/review \
  -H 'Content-Type: application/json' \
```

```
  -H "x-correlation-id: $CID" \
  -d '{
    "orderId": "ORD-REV-001",
    "amount": 110000,
    "reason": "HIGH_RISK",
    "customerId": "CUST-300",
    "channel": "web"
  }'
```

What happens beneath:

- MI review flow ensures correlation ID
- logs review receipt
- calls CRM synchronously
- returns REVIEW_TRIGGERED response with CRM details

Expected result:

- HTTP 200
- JSON containing:
    - status: "REVIEW_TRIGGERED"
    - correlationId
    - orderId
    - amount
    - crm

Note:

- the review payload is not heavily validated; it mainly extracts orderId and amount

2.11 POST /review — CRM failure during review

Force CRM failure:

```shell
curl -s -X POST http://localhost:8081/admin/crm/fail \
  -H 'Content-Type: application/json' \
  -d '{"enabled": true}'
```

Then:

```shell
curl -s -i -X POST http://localhost:8290/review \
  -H 'Content-Type: application/json' \
  -d '{
    "orderId": "ORD-REV-FAIL",
    "amount": 70000
  }'
```

What happens beneath:

- review flow calls CRM
- CRM fails
- CommonFaultHandler returns integration error JSON

Expected result:

- HTTP error response, likely 502/503 depending on what MI surfaces into the fault path
- JSON fault body with correlation ID

Reset CRM after test.

3) Siddhi / SI cookbook

The Siddhi app exposes one HTTP source endpoint.

3.1 Send normal order event to SI

```shell
curl -s -X POST http://localhost:8007/OrderEvents \
  -H 'Content-Type: application/json' \
  -d '{
    "event": {
      "orderId": "ORD-SI-001",
      "amount": 5000,
      "customerId": "CUST-S1",
      "channel": "web",
      "correlationId": "si-cid-001"
    }
  }'
```

What happens beneath:

- Siddhi ingests event
- normalizes fields
- amount is below anomaly thresholds

- no anomaly is emitted

Expected result:

- HTTP acceptance from Siddhi source
- no downstream MI review or BI anomaly event should be triggered

3.2 Trigger immediate single high-value anomaly

```shell
curl -s -X POST http://localhost:8007/OrderEvents \
  -H 'Content-Type: application/json' \
  -d '{
    "event": {
      "orderId": "ORD-SI-HIGH-001",
      "amount": 150000,
      "customerId": "CUST-S2",
      "channel": "mobile",
      "correlationId": "si-cid-high-001"
    }
  }'
```

What happens beneath:

- Siddhi sees amount >= 100000
- emits SINGLE_HIGH_VALUE anomaly
- anomaly is:
    - logged
    - posted to MI /review
    - posted to BI /bi/events/anomaly

Expected result:

- SI accepts the request
- downstream side effects:
    - MI review flow runs
    - BI anomaly buffer increments
- later BI reports will reflect this anomaly

3.3 Trigger burst anomaly with 3 events

Send these three within 2 minutes:

```shell
curl -s -X POST http://localhost:8007/OrderEvents \
  -H 'Content-Type: application/json' \
  -d '{
    "event": {
      "orderId": "ORD-SI-B1",
      "amount": 12000,
      "customerId": "CUST-BURST",
      "channel": "web",
      "correlationId": "burst-1"
    }
  }'

curl -s -X POST http://localhost:8007/OrderEvents \
  -H 'Content-Type: application/json' \
  -d '{
    "event": {
      "orderId": "ORD-SI-B2",
      "amount": 13000,
      "customerId": "CUST-BURST",
      "channel": "web",
      "correlationId": "burst-2"
    }
  }'

curl -s -X POST http://localhost:8007/OrderEvents \
  -H 'Content-Type: application/json' \
  -d '{
    "event": {
      "orderId": "ORD-SI-B3",
      "amount": 14000,
      "customerId": "CUST-BURST",
      "channel": "web",
      "correlationId": "burst-3"
    }
  }'
```

What happens beneath:

- all three events are normalized
- all match amount >= 10000
- at the 2-minute batch window evaluation, Siddhi groups by customerId, channel
- since count is 3, emits one BURST_HIGH_VALUE anomaly
- anomaly goes to MI review and BI ingestion

Expected result:

- not necessarily immediate anomaly response from SI itself
- after the batch closes, one burst anomaly is generated with:
    - anomalyType: "BURST_HIGH_VALUE"
    - eventCount >= 3

Important:

- because it is a timeBatch(2 min), this anomaly may appear when the current batch completes, not instantly on the third POST

3.4 Send event with missing optional fields

```shell
curl -s -X POST http://localhost:8007/OrderEvents \
  -H 'Content-Type: application/json' \
  -d '{
    "event": {
      "orderId": "",
      "amount": 20000,
      "customerId": "",
      "channel": "",
      "correlationId": ""
    }
  }'
```

What happens beneath:

- normalization query substitutes:
    - orderId -> "UNKNOWN"
    - customerId -> "ANON"
    - channel -> "unknown"
    - correlationId -> orderId

Expected result:

- event is accepted
- depending on amount and window, it may contribute to anomalies under normalized values

4) BI cookbook
4.1 BI health

```shell
curl -s http://localhost:9090/bi/health
```

What happens beneath:

- returns a simple health JSON with current time

Expected result:

- status: "OK"
- runtime: "BI"

4.2 Ingest anomaly directly into BI

```shell
curl -s -X POST http://localhost:9090/bi/events/anomaly \
  -H 'Content-Type: application/json' \
  -H "x-correlation-id: $CID" \
  -d '{
    "anomalyType": "SINGLE_HIGH_VALUE",
    "orderId": "ORD-BI-ANOM-001",
    "amount": 120000,
    "customerId": "CUST-BI",
    "channel": "web",
    "correlationId": "bi-anom-001",
    "eventCount": 1,
    "windowStartTime": 1740000000
  }'
```

What happens beneath:

- BI normalizes payload into AnomalyEvent
- stores it in in-memory bounded buffer
- logs ingestion
- returns accepted

Expected result:

- HTTP 202
- JSON:
    - status: "ACCEPTED"
    - buffered count incremented

4.3 Get latest JSON report

```shell
curl -s http://localhost:9090/bi/reports/latest
```

What happens beneath:

- BI reads /data/reports/latest.json

Expected result:

- if reconciliation has already run:
    - HTTP 200
    - JSON report containing:
        - anomaly stats
        - top customers
        - backend state
        - MI readiness/liveness
- if not yet generated:
    - HTTP 404
    - No report generated yet

Since your scheduler starts shortly after startup and runs every 300 seconds by default, wait a bit after boot if needed.

4.4 Get latest CSV report

```shell
curl -s http://localhost:9090/bi/reports/latestCsv
```

What happens beneath:

- BI reads /data/reports/latest.csv

Expected result:

- HTTP 200 with CSV if a report exists
- otherwise HTTP 404 with plain text message

4.5 BI AI review — happy path with OpenAI enabled

```shell
curl -s -X POST http://localhost:9090/bi/ai/review \
```

```
  -H 'Content-Type: application/json' \
  -H "x-correlation-id: $CID" \
  -d '{
    "orderId": "ORD-AI-001",
    "amount": 15000,
    "customerId": "CUST-AI",
    "channel": "web"
  }'
```

What happens beneath:

- BI validates the order
- calls CRM directly on mock backend
- looks up anomaly stats for customer/channel over last 15 minutes
- calls OpenAI Responses API with structured schema
- if model returns a valid decision:
    - response mode is OPENAI_AGENT
- if action is TRIGGER_MI_REVIEW, BI also calls MI /review

Expected result:

- HTTP 200
- JSON with:
    - status: "RECOMMENDATION_READY"
    - risk
    - recommendedAction
    - rationale
    - crm
    - miReview maybe present
    - decisionMode: "OPENAI_AGENT"

4.6 BI AI review — fallback rules path when OpenAI key is absent

If OPENAI_API_KEY is empty, run:

```shell
curl -s -X POST http://localhost:9090/bi/ai/review \
  -H 'Content-Type: application/json' \
  -d '{
    "orderId": "ORD-AI-FALLBACK-001",
    "amount": 105000,
    "customerId": "CUST-AI2",
    "channel": "web"
```

```
}'
```

What happens beneath:

- OpenAI call fails because key is not set
- BI logs warning
- fallback rules execute:
  - amount >= 100000 => HIGH
  - action => TRIGGER_MI_REVIEW
- BI then invokes MI review

Expected result:

- HTTP 200
- JSON with:
  - decisionMode: "FALLBACK_RULES"
  - risk: "HIGH"
  - recommendedAction: "TRIGGER_MI_REVIEW"
  - miReview populated

4.7 BI AI review — medium risk fallback

```shell
curl -s -X POST http://localhost:9090/bi/ai/review \
  -H 'Content-Type: application/json' \
  -d '{
    "orderId": "ORD-AI-FALLBACK-002",
    "amount": 15000,
    "customerId": "CUST-AI3",
    "channel": "mobile"
  }'
```

What happens beneath:

- fallback logic sees amount >= 10000 but < 100000
- returns medium risk unless OpenAI overrides it

Expected result:

- likely:
  - risk: "MEDIUM"
  - recommendedAction: "ALLOW_BUT_MONITOR"

4.8 BI AI review — invalid payload

```shell
curl -s -i -X POST http://localhost:9090/bi/ai/review \
  -H 'Content-Type: application/json' \
  -d '{
    "orderId": "",
    "amount": 0
  }'
```

What happens beneath:

- BI validates input before CRM/OpenAI call

Expected result:

- HTTP 400
- JSON:
    - status: "INVALID"
    - explanation about non-empty orderId and amount > 0

4.9 Agent chat

```shell
curl -s -X POST http://localhost:9090/agente/chat \
  -H 'Content-Type: application/json' \
  -d '{
    "message": "Summarize the BI anomaly handling flow in one paragraph",
    "sessionId": "demo-session-1"
  }'
```

What happens beneath:

- BI agent service passes the message to agenteAgent.run(...)
- short-term memory is used per session
- response comes from configured OpenAI model provider

Expected result:

- chat response JSON with a message

Important:

- this likely requires a valid OpenAI API key and correct Ballerina AI package behavior at runtime
-

5) End-to-end scenario cookbook

These are the most useful full-flow tests.

5.1 End-to-end happy path: v1 order with async shipping

```shell
curl -s -X POST http://localhost:8290/v1/orders \
  -H 'Content-Type: application/json' \
  -H 'x-correlation-id: e2e-v1-001' \
  -d '{
    "orderId": "ORD-E2E-V1",
    "amount": 18000,
    "customerId": "CUST-E2E",
    "channel": "web"
  }'
```

What happens beneath:

- MI validates
- ERP sync call
- CRM sync call
- immediate response built
- shipping message stored to ShippingMS
- message processor later invokes shipping slow endpoint

Expected result:

- immediate response with ERP and CRM blocks
- shipping happens in background; inspect logs to confirm delivery

5.2 End-to-end v2 non-premium path

```shell
curl -s -X POST http://localhost:8290/v2/orders \
  -H 'Content-Type: application/json' \
  -H 'x-correlation-id: e2e-v2-001' \
  -d '{
    "orderId": "ORD-E2E-V2-LOW",
    "amount": 3000,
    "customerId": "CUST-E2E2",
    "channel": "mobile"
  }'
```

What happens beneath:

- MI sync ERP + CRM only
- premium check fails
- shipping skipped

Expected result:

- accepted response
- no async shipping

5.3 End-to-end SI anomaly → MI review + BI ingest

```shell
curl -s -X POST http://localhost:8007/OrderEvents \
  -H 'Content-Type: application/json' \
  -d '{
    "event": {
      "orderId": "ORD-E2E-SI-1",
      "amount": 125000,
      "customerId": "CUST-ANOM",
      "channel": "web",
      "correlationId": "e2e-si-001"
    }
  }'
```

Then check BI health/report later:

```shell
curl -s http://localhost:9090/bi/reports/latest
```

What happens beneath:

- SI emits SINGLE_HIGH_VALUE
- SI posts anomaly to MI review
- SI posts anomaly to BI event ingestion
- BI stores anomaly
- later scheduled report includes it

Expected result:

- anomaly stored in BI
- MI review triggered
- future report stats increment

5.4 End-to-end shipping retry story

Force shipping failure:

```shell
curl -s -X POST http://localhost:8081/admin/shipping/fail \
  -H 'Content-Type: application/json' \
  -d '{"enabled": true}'
```

Send premium v1 order:

```shell
curl -s -X POST http://localhost:8290/v1/orders \
  -H 'Content-Type: application/json' \
  -H 'x-correlation-id: e2e-ship-retry-1' \
  -d '{
    "orderId": "ORD-SHIP-RETRY",
    "amount": 22000,
    "customerId": "CUST-RETRY",
    "channel": "web"
  }'
```

Then re-enable shipping:

```shell
curl -s -X POST http://localhost:8081/admin/shipping/fail \
  -H 'Content-Type: application/json' \
  -d '{"enabled": false}'
```

What happens beneath:

- MI stores shipping request in message store
- background processor tries sending to shipping
- shipping fails while fail mode is on
- processor retries according to its settings
- once shipping is healthy again, later retry should succeed

Expected result:

- original /v1/orders still returns accepted
- shipping delivery is eventually successful after retries
- this is best verified via MI/mock logs

6) Suggested verification commands

These are useful while testing.

Follow all logs

```shell
docker compose logs -f mock-backend mi si bi
```

You should see:

- correlation IDs moving across services
- SI anomaly logs
- BI anomaly ingestion logs
- MI message processor logs for shipping retries/faults

Check current backend state any time

```shell
curl -s http://localhost:8081/admin/state
```

Check whether BI has generated reports

```shell
curl -s http://localhost:9090/bi/reports/latest
curl -s http://localhost:9090/bi/reports/latestCsv
```